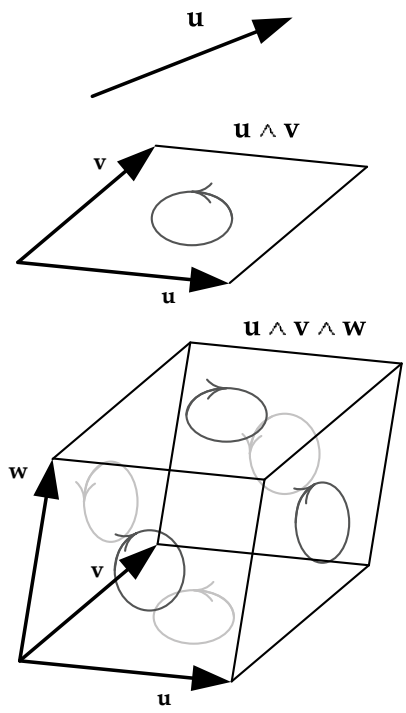

Figure 5.15. In the exterior algebra (Grassman algebra), a single wedge product yields a pseudovector or bivector, and two wedge products yield a pseudoscalar or trivector.

dovectors (also known as *axial vectors*, *covariant vectors*, *bivectors* or *2-blades*). The surface normal of a triangle (which is calculated using a cross product) is also a pseudovector.

It's pretty interesting to note that the cross product ($\mathbf{A} \times \mathbf{B}$), the scalar triple product ($\mathbf{A} \cdot (\mathbf{B} \times \mathbf{C})$) and the determinant of a matrix are all inter-related, and pseudovectors lie at the heart of it all. Mathematicians have come up with a set of algebraic rules, called an *exterior algebra* or *Grassman algebra*, which describe how vectors and pseudovectors work and allow us to calculate areas of parallelograms (in 2D), volumes of parallelepipeds (in 3D), and so on in higher dimensions.

We won't get into all the details here, but the basic idea of Grassman algebra is to introduce a special kind of vector product known as the *wedge product*, denoted $\mathbf{A} \wedge \mathbf{B}$. A pairwise wedge product yields a pseudovector and is equivalent to a cross product, which also represents the *signed area* of the parallelogram formed by the two vectors (where the sign tells us whether we're rotating from $\mathbf{A}$ to $\mathbf{B}$ or vice versa). Doing two wedge products in a row, $\mathbf{A} \wedge \mathbf{B} \wedge \mathbf{C}$,

is equivalent to the scalar triple product $\mathbf{A} \cdot (\mathbf{B} \times \mathbf{C})$ and produces another strange mathematical object known as a *pseudoscalar* (also known as a *trivector* or a *3-blade*), which can be interpreted as the *signed volume* of the parallelepiped formed by the three vectors (see Figure 5.15). This extends into higher dimensions as well.

What does all this mean for us as game programmers? Not too much. All we really need to keep in mind is that some vectors in our code are actually pseudovectors, so that we can transform them properly when changing handedness, for example. Of course if you really want to geek out, you can impress your friends by talking about *exterior algebras* and *wedge products* and explaining how cross products aren't really vectors. Which might make you look cool at your next social engagement …or not.

For more information, see http://en.wikipedia.org/wiki/Pseudovector, http://en.wikipedia.org/wiki/Exterior_algebra, and http://www.terathon.com/gdc12_lengyel.pdf.

5.2.5 Linear Interpolation of Points and Vectors

In games, we often need to find a vector that is midway between two known vectors. For example, if we want to smoothly animate an object from point **A** to point **B** over the course of two seconds at 30 frames per second, we would need to find 60 intermediate positions between **A** and **B**.

A *linear interpolation* is a simple mathematical operation that finds an intermediate point between two known points. The name of this operation is often shortened to LERP. The operation is defined as follows, where β ranges from 0 to 1 inclusive:

$$\begin{aligned}\mathbf{L} &= \text{LERP}(\mathbf{A}, \mathbf{B}, \beta) = (1-\beta)\mathbf{A} + \beta\mathbf{B} \\ &= \left[(1-\beta)A_x + \beta B_x, \quad (1-\beta)A_y + \beta B_y, \quad (1-\beta)A_z + \beta B_z\right]\end{aligned}$$

Geometrically, $\mathbf{L} = \text{LERP}(\mathbf{A}, \mathbf{B}, \beta)$ is the position vector of a point that lies β percent of the way along the line segment from point **A** to point **B**, as shown in Figure 5.16. Mathematically, the LERP function is just a *weighted average* of the two input vectors, with weights $(1-\beta)$ and β, respectively. Notice that the weights always add to 1, which is a general requirement for any weighted average.

5.3 Matrices

A *matrix* is a rectangular array of $m \times n$ scalars. Matrices are a convenient way of representing linear transformations such as translation, rotation and scale.

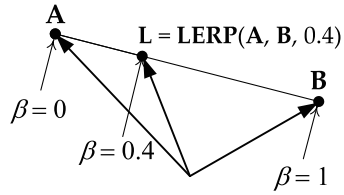

Figure 5.16. Linear interpolation (LERP) between points **A** and **B**, with $\beta = 0.4$.

A matrix **M** is usually written as a grid of scalars M_{rc} enclosed in square brackets, where the subscripts r and c represent the row and column indices of the entry, respectively. For example, if **M** is a 3×3 matrix, it could be written as follows:

$$\mathbf{M} = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix}.$$

We can think of the rows and/or columns of a 3×3 matrix as 3D vectors. When all of the row and column vectors of a 3×3 matrix are of unit magnitude, we call it a *special orthogonal* matrix. This is also known as an *isotropic* matrix, or an *orthonormal* matrix. Such matrices represent pure rotations.

Under certain constraints, a 4×4 matrix can represent arbitrary 3D *transformations*, including *translations*, *rotations*, and changes in *scale*. These are called *transformation matrices*, and they are the kinds of matrices that will be most useful to us as game engineers. The transformations represented by a matrix are applied to a point or vector via matrix multiplication. We'll investigate how this works below.

An *affine* matrix is a 4×4 transformation matrix that preserves parallelism of lines and relative distance ratios, but not necessarily absolute lengths and angles. An affine matrix is any combination of the following operations: rotation, translation, scale and/or shear.

5.3.1 Matrix Multiplication

The product **P** of two matrices **A** and **B** is written $\mathbf{P} = \mathbf{AB}$. If **A** and **B** are transformation matrices, then the product **P** is another transformation matrix that performs *both* of the original transformations. For example, if **A** is a scale matrix and **B** is a rotation, the matrix **P** would both scale *and* rotate the points or vectors to which it is applied. This is particularly useful in game programming, because we can precalculate a single matrix that performs a whole sequence of transformations and then apply all of those transformations to a large number of vectors efficiently.

To calculate a matrix product, we simply take dot products between the rows of the $n_A \times m_A$ matrix $\mathbf{A}$ and the columns of the $n_B \times m_B$ matrix $\mathbf{B}$. Each dot product becomes one component of the resulting matrix $\mathbf{P}$. The two matrices can be multiplied as long as the *inner dimensions* are equal (i.e., $m_A = n_B$). For example, if $\mathbf{A}$ and $\mathbf{B}$ are 3×3 matrices, then $\mathbf{P} = \mathbf{AB}$ may be expressed as follows:

$$\begin{aligned}\mathbf{P} &= \begin{bmatrix} P_{11} & P_{12} & P_{13} \\ P_{21} & P_{22} & P_{23} \\ P_{31} & P_{32} & P_{33} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A}_{\text{row1}} \cdot \mathbf{B}_{\text{col1}} & \mathbf{A}_{\text{row1}} \cdot \mathbf{B}_{\text{col2}} & \mathbf{A}_{\text{row1}} \cdot \mathbf{B}_{\text{col3}} \\ \mathbf{A}_{\text{row2}} \cdot \mathbf{B}_{\text{col1}} & \mathbf{A}_{\text{row2}} \cdot \mathbf{B}_{\text{col2}} & \mathbf{A}_{\text{row2}} \cdot \mathbf{B}_{\text{col3}} \\ \mathbf{A}_{\text{row3}} \cdot \mathbf{B}_{\text{col1}} & \mathbf{A}_{\text{row3}} \cdot \mathbf{B}_{\text{col2}} & \mathbf{A}_{\text{row3}} \cdot \mathbf{B}_{\text{col3}} \end{bmatrix}.\end{aligned}$$

Matrix multiplication is not commutative. In other words, the order in which matrix multiplication is done matters:

$$\mathbf{AB} \neq \mathbf{BA}$$

We'll see exactly why this matters in Section 5.3.2.

Matrix multiplication is often called *concatenation*, because the product of n transformation matrices is a matrix that concatenates, or chains together, the original sequence of transformations in the order the matrices were multiplied.

5.3.2 Representing Points and Vectors as Matrices

Points and vectors can be represented as *row matrices* $(1 \times n)$ or *column matrices* $(n \times 1)$, where n is the dimension of the space we're working with (usually 2 or 3). For example, the vector $\mathbf{v} = (3, 4, -1)$ can be written either as

$$\mathbf{v}_1 = \begin{bmatrix} 3 & 4 & -1 \end{bmatrix}$$

or as

$$\mathbf{v}_2 = \begin{bmatrix} 3 \\ 4 \\ -1 \end{bmatrix} = \mathbf{v}_1^{\mathrm{T}}.$$

Here, the superscripted T represents matrix *transposition* (see Section 5.3.5).

The choice between column and row vectors is a completely arbitrary one, but it does affect the order in which matrix multiplications are written. This happens because when multiplying matrices, the inner dimensions of the two matrices must be equal, so

- to multiply a $1 \times n$ row vector by an $n \times n$ matrix, the vector must appear to the *left* of the matrix ($\mathbf{v}'_{1\times n} = \mathbf{v}_{1\times n}\,\mathbf{M}_{n\times n}$), whereas
- to multiply an $n \times n$ matrix by an $n \times 1$ column vector, the vector must appear to the *right* of the matrix ($\mathbf{v}'_{n\times 1} = \mathbf{M}_{n\times n}\,\mathbf{v}_{n\times 1}$).

If multiple transformation matrices **A**, **B** and **C** are applied in order to a vector **v**, the transformations "read" from *left to right* when using *row* vectors, but from *right to left* when using *column* vectors. The easiest way to remember this is to realize that the matrix *closest* to the vector is applied first. This is illustrated by the parentheses below:

$$\begin{aligned} \mathbf{v}' &= (((\mathbf{v}\mathbf{A})\mathbf{B})\mathbf{C}) && \text{Row vectors: read left-to-right;} \\ \mathbf{v}'^{\mathrm{T}} &= (\mathbf{C}^{\mathrm{T}}(\mathbf{B}^{\mathrm{T}}(\mathbf{A}^{\mathrm{T}}\mathbf{v}^{\mathrm{T}}))) && \text{Column vectors: read right-to-left.} \end{aligned}$$

In this book we'll adopt the *row vector convention*, because the left-to-right order of transformations is most intuitive to read for English-speaking people. That said, be very careful to check which convention is used by your game engine, and by other books, papers or web pages you may read. You can usually tell by seeing whether vector-matrix multiplications are written with the vector on the left (for row vectors) or the right (for column vectors) of the matrix. When using column vectors, you'll need to *transpose* all the matrices shown in this book.

5.3.3 The Identity Matrix

The *identity matrix* is a matrix that, when multiplied by any other matrix, yields the very same matrix. It is usually represented by the symbol **I**. The identity matrix is always a square matrix with 1's along the diagonal and 0's everywhere else:

$$\mathbf{I}_{3\times 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix};$$

$$\mathbf{A}\mathbf{I} = \mathbf{I}\mathbf{A} \equiv \mathbf{A}.$$

5.3.4 Matrix Inversion

The *inverse* of a matrix **A** is another matrix (denoted $\mathbf{A}^{-1}$) that *undoes* the effects of matrix **A**. So, for example, if **A** rotates objects by 37 degrees about the z-axis, then $\mathbf{A}^{-1}$ will rotate by -37 degrees about the z-axis. Likewise, if **A** scales objects to be twice their original size, then $\mathbf{A}^{-1}$ scales objects to be half-sized. When a matrix is multiplied by its own inverse, the result is *always* the identity matrix, so $\mathbf{A}(\mathbf{A}^{-1}) \equiv (\mathbf{A}^{-1})\mathbf{A} \equiv \mathbf{I}$. Not all matrices have

inverses. However, all *affine* matrices (combinations of pure rotations, translations, scales and shears) do have inverses. Gaussian elimination or lower-upper (LU) decomposition can be used to find the inverse, if one exists.

Since we'll be dealing with matrix multiplication a lot, it's important to note here that the inverse of a sequence of concatenated matrices can be written as the *reverse concatenation* of the individual matrices' inverses. For example,

$$(\mathbf{ABC})^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}.$$

5.3.5 Transposition

The *transpose* of a matrix $\mathbf{M}$ is denoted $\mathbf{M}^{\mathrm{T}}$. It is obtained by reflecting the entries of the original matrix across its diagonal. In other words, the rows of the original matrix become the columns of the transposed matrix, and vice versa:

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix}.$$

The transpose is useful for a number of reasons. For one thing, the inverse of an orthonormal (pure rotation) matrix is exactly equal to its transpose—which is good news, because it's much cheaper to transpose a matrix than it is to find its inverse in general. Transposition can also be important when moving data from one math library to another, because some libraries use column vectors while others expect row vectors. The matrices used by a row-vector–based library will be *transposed* relative to those used by a library that employs the column vector convention.

As with the inverse, the transpose of a sequence of concatenated matrices can be rewritten as the reverse concatenation of the individual matrices' transposes. For example,

$$(\mathbf{ABC})^{\mathrm{T}} = \mathbf{C}^{\mathrm{T}}\mathbf{B}^{\mathrm{T}}\mathbf{A}^{\mathrm{T}}.$$

This will prove useful when we consider how to apply transformation matrices to points and vectors.

5.3.6 Homogeneous Coordinates

You may recall from high-school algebra that a 2×2 matrix can represent a rotation in two dimensions. To rotate a vector $\mathbf{r}$ through an angle of ϕ degrees (where positive rotations are counterclockwise), we can write

$$\begin{bmatrix} r'_x & r'_y \end{bmatrix} = \begin{bmatrix} r_x & r_y \end{bmatrix} \begin{bmatrix} \cos\phi & \sin\phi \\ -\sin\phi & \cos\phi \end{bmatrix}.$$

It's probably no surprise that rotations in three dimensions can be represented by a 3×3 matrix. The two-dimensional example above is really just a three-dimensional rotation about the z-axis, so we can write

$$\begin{bmatrix} r'_x & r'_y & r'_z \end{bmatrix} = \begin{bmatrix} r_x & r_y & r_z \end{bmatrix} \begin{bmatrix} \cos\phi & \sin\phi & 0 \\ -\sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The question naturally arises: Can a 3×3 matrix be used to represent *translations?* Sadly, the answer is no. The result of translating a point $\mathbf{r}$ by a translation $\mathbf{t}$ requires adding the components of $\mathbf{t}$ to the components of $\mathbf{r}$ individually:

$$\mathbf{r} + \mathbf{t} = \begin{bmatrix} (r_x + t_x) & (r_y + t_y) & (r_z + t_z) \end{bmatrix}.$$

Matrix multiplication involves multiplication and addition of matrix elements, so the idea of using multiplication for translation seems promising. But, unfortunately, there is no way to arrange the components of $\mathbf{t}$ within a 3×3 matrix such that the result of multiplying it with the column vector $\mathbf{r}$ yields sums like $(r_x + t_x)$.

The good news is that we *can* obtain sums like this if we use a 4×4 matrix. What would such a matrix look like? Well, we know that we don't want any rotational effects, so the upper 3×3 should contain an identity matrix. If we arrange the components of $\mathbf{t}$ across the bottom-most row of the matrix and set the fourth element of the $\mathbf{r}$ vector (usually called w) equal to 1, then taking the dot product of the vector $\mathbf{r}$ with column 1 of the matrix will yield $(1 \cdot r_x) + (0 \cdot r_y) + (0 \cdot r_z) + (t_x \cdot 1)$, which is exactly what we want. If the bottom right-hand corner of the matrix contains a 1 and the rest of the fourth column contains zeros, then the resulting vector will also have a 1 in its w component. Here's what the final 4×4 translation matrix looks like:

$$\begin{aligned} \mathbf{r} + \mathbf{t} &= \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix} \\ &= \begin{bmatrix} (r_x + t_x) & (r_y + t_y) & (r_z + t_z) & 1 \end{bmatrix}. \end{aligned}$$

When a point or vector is extended from three dimensions to four in this manner, we say that it has been written in *homogeneous coordinates*. A point in homogeneous coordinates always has $w = 1$. Most of the 3D matrix math done by game engines is performed using 4×4 matrices with four-element points and vectors written in homogeneous coordinates.

5.3.6.1 Transforming Direction Vectors

Mathematically, points (position vectors) and direction vectors are treated in subtly different ways. When transforming a point by a matrix, the translation, rotation and scale of the matrix are all applied to the point. But when transforming a direction by a matrix, the *translational* effects of the matrix are ignored. This is because direction vectors have no translation per se—applying a translation to a direction would alter its magnitude, which is usually not what we want.

In homogeneous coordinates, we achieve this by defining points to have their w components equal to one, while direction vectors have their w components equal to *zero*. In the example below, notice how the $w = 0$ component of the vector $\mathbf{v}$ multiplies with the $\mathbf{t}$ vector in the matrix, thereby eliminating translation in the final result:

$$\begin{bmatrix} \mathbf{v} & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U} & 0 \\ \mathbf{t} & 1 \end{bmatrix} = \begin{bmatrix} (\mathbf{vU} + 0\mathbf{t}) & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{vU} & 0 \end{bmatrix}.$$

Technically, a point in homogeneous (four-dimensional) coordinates can be converted into non-homogeneous (three-dimensional) coordinates by dividing the x, y and z components by the w component:

$$\begin{bmatrix} x & y & z & w \end{bmatrix} \equiv \begin{bmatrix} \frac{x}{w} & \frac{y}{w} & \frac{z}{w} \end{bmatrix}.$$

This sheds some light on why we set a point's w component to one and a vector's w component to zero. Dividing by $w = 1$ has no effect on the coordinates of a point, but dividing a pure direction vector's components by $w = 0$ would yield infinity. A point at infinity in 4D can be rotated but not translated, because no matter what translation we try to apply, the point will remain at infinity. So in effect, a pure direction vector in three-dimensional space acts like a point at infinity in four-dimensional homogeneous space.

5.3.7 Atomic Transformation Matrices

Any affine transformation matrix can be created by simply concatenating a sequence of 4×4 matrices representing pure translations, pure rotations, pure scale operations and/or pure shears. These atomic transformation building blocks are presented below. (We'll omit shear from these discussions, as it tends to be used only rarely in games.)

Notice that all affine 4×4 transformation matrices can be partitioned into four components:

$$\mathbf{M}_{\text{affine}} = \begin{bmatrix} \mathbf{U}_{3\times 3} & 0_{3\times 1} \\ \mathbf{t}_{1\times 3} & 1 \end{bmatrix}.$$

- the upper 3×3 matrix $\mathbf{U}$, which represents the rotation and/or scale,
- a 1×3 translation vector $\mathbf{t}$,
- a 3×1 vector of zeros $0 = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^{\mathrm{T}}$, and
- a scalar 1 in the bottom-right corner of the matrix.

When a point is multiplied by a matrix that has been partitioned like this, the result is as follows:

$$\begin{bmatrix} \mathbf{r}'_{1\times 3} & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{r}_{1\times 3} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{U}_{3\times 3} & 0_{3\times 1} \\ \mathbf{t}_{1\times 3} & 1 \end{bmatrix} = \begin{bmatrix} (\mathbf{r}\mathbf{U} + \mathbf{t}) & 1 \end{bmatrix}.$$

5.3.7.1 Translation

The following matrix translates a point by the vector $\mathbf{t}$:

$$\begin{aligned} \mathbf{r} + \mathbf{t} &= \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix} \\ &= \begin{bmatrix} (r_x + t_x) & (r_y + t_y) & (r_z + t_z) & 1 \end{bmatrix}, \end{aligned} \tag{5.3}$$

or in partitioned shorthand:

$$\begin{bmatrix} \mathbf{r} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I} & 0 \\ \mathbf{t} & 1 \end{bmatrix} = \begin{bmatrix} (\mathbf{r} + \mathbf{t}) & 1 \end{bmatrix}.$$

To invert a pure translation matrix, simply negate the vector $\mathbf{t}$ (i.e., negate t_x, t_y and t_z).

5.3.7.2 Rotation

All 4×4 pure rotation matrices have the form

$$\begin{bmatrix} \mathbf{r} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R} & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{r}\mathbf{R} & 1 \end{bmatrix}.$$

The $\mathbf{t}$ vector is zero, and the upper 3×3 matrix $\mathbf{R}$ contains cosines and sines of the rotation angle, measured in radians.

The following matrix represents rotation about the x-axis by an angle ϕ.

$$\mathrm{rotate}_x(\mathbf{r}, \phi) = \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\phi & \sin\phi & 0 \\ 0 & -\sin\phi & \cos\phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{5.4}$$

The matrix below represents rotation about the y-axis by an angle θ. (Notice that this one is *transposed* relative to the other two—the positive and negative sine terms have been reflected across the diagonal.)

$$\text{rotate}_y(\mathbf{r}, \theta) = \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & 0 & -\sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{5.5}$$

The following matrix represents rotation about the z-axis by an angle γ:

$$\text{rotate}_z(\mathbf{r}, \gamma) = \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} \cos\gamma & \sin\gamma & 0 & 0 \\ -\sin\gamma & \cos\gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{5.6}$$

Here are a few observations about these matrices:

- The 1 within the upper 3×3 always appears on the axis we're rotating about, while the sine and cosine terms are off-axis.
- Positive rotations go from x to y (about z), from y to z (about x) and from z to x (about y). The z to x rotation "wraps around," which is why the rotation matrix about the y-axis is transposed relative to the other two. (Use the right-hand or left-hand rule to remember this.)
- The inverse of a pure rotation is just its transpose. This works because inverting a rotation is equivalent to rotating by the negative angle. You may recall that $\cos(-\theta) = \cos(\theta)$ while $\sin(-\theta) = -\sin(\theta)$, so negating the angle causes the two sine terms to effectively switch places, while the cosine terms stay put.

5.3.7.3 Scale

The following matrix scales the point $\mathbf{r}$ by a factor of s_x along the x-axis, s_y along the y-axis and s_z along the z-axis:

$$\begin{aligned} \mathbf{rS} &= \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} s_x r_x & s_y r_y & s_z r_z & 1 \end{bmatrix}, \end{aligned} \tag{5.7}$$

or in partitioned shorthand:

$$\begin{bmatrix} \mathbf{r} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{S}_{3\times3} & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{rS}_{3\times3} & 1 \end{bmatrix}.$$

Here are some observations about this kind of matrix:

- To invert a scaling matrix, simply substitute s_x, s_y and s_z with their reciprocals (i.e., $1/s_x$, $1/s_y$ and $1/s_z$).
- When the scale factor along all three axes is the same ($s_x = s_y = s_z$), we call this *uniform scale*. Spheres remain spheres under uniform scale, whereas under nonuniform scale they become ellipsoids. To keep the mathematics of bounding sphere checks simple and fast, many game engines impose the restriction that only uniform scale may be applied to renderable geometry or collision primitives.
- When a uniform scale matrix $\mathbf{S}_u$ and a rotation matrix $\mathbf{R}$ are concatenated, the order of multiplication is unimportant (i.e., $\mathbf{S}_u\mathbf{R} = \mathbf{R}\mathbf{S}_u$). This only works for *uniform* scale!

5.3.8 4×3 Matrices

The rightmost column of an affine 4×4 matrix always contains the vector $\begin{bmatrix}0 & 0 & 0 & 1\end{bmatrix}^{\mathrm{T}}$. As such, game programmers often omit the fourth column to save memory. You'll encounter 4×3 affine matrices frequently in game math libraries.

5.3.9 Coordinate Spaces

We've seen how to apply transformations to points and direction vectors using 4×4 matrices. We can extend this idea to rigid objects by realizing that such an object can be thought of as an infinite collection of points. Applying a transformation to a rigid object is like applying that same transformation to every point within the object. For example, in computer graphics an object is usually represented by a mesh of triangles, each of which has three vertices represented by points. In this case, the object can be transformed by applying a transformation matrix to all of its vertices in turn.

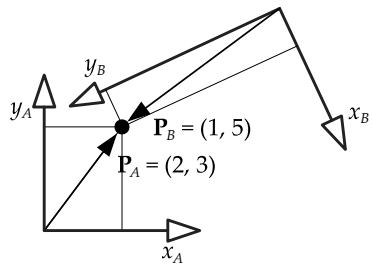

Figure 5.17. Position vectors for the point P relative to different coordinate axes.

We said above that a point is a vector whose tail is fixed to the origin of some coordinate system. This is another way of saying that a point (position vector) is always expressed *relative* to a set of coordinate axes. The triplet of numbers representing a point changes numerically whenever we select a new set of coordinate axes. In Figure 5.17, we see a point P represented by two different position vectors—the vector $\mathbf{P}_A$ gives the position of **P** relative to the "A" axes, while the vector $\mathbf{P}_B$ gives the position of that same point relative to a different set of axes "B."

In physics, a set of coordinate axes represents a frame of reference, so we sometimes refer to a set of axes as a *coordinate frame* (or just a *frame*). People in the game industry also use the term *coordinate space* (or simply *space*) to refer to a set of coordinate axes. In the following sections, we'll look at a few of the most common coordinate spaces used in games and computer graphics.

5.3.9.1 Model Space

When a triangle mesh is created in a tool such as Maya or 3DStudioMAX, the positions of the triangles' vertices are specified relative to a Cartesian coordinate system, which we call *model space* (also known as *object space* or *local space*). The model-space origin is usually placed at a central location within the object, such as at its center of mass, or on the ground between the feet of a humanoid or animal character.

Most game objects have an inherent directionality. For example, an airplane has a nose, a tail fin and wings that correspond to the front, up and left/right directions. The model-space axes are usually aligned to these natural directions on the model, and they're given intuitive names to indicate their directionality as illustrated in Figure 5.18.

- *Front*. This name is given to the axis that points in the direction that the object naturally travels or faces. In this book, we'll use the symbol **F** to

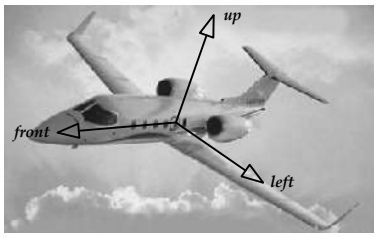

Figure 5.18. One possible choice of the model-space front, left and up axis basis vectors for an airplane.

refer to a unit basis vector along the front axis.

- *Up*. This name is given to the axis that points towards the top of the object. The unit basis vector along this axis will be denoted **U**.
- *Left or right*. The name "left" or "right" is given to the axis that points toward the left or right side of the object. Which name is chosen depends on whether your game engine uses left-handed or right-handed coordinates. The unit basis vector along this axis will be denoted **L** or **R**, as appropriate.

The mapping between the $(front, up, left)$ labels and the (x, y, z) axes is completely arbitrary. A common choice when working with right-handed axes is to assign the label *front* to the positive z-axis, the label *left* to the positive x-axis and the label *up* to the positive y-axis (or in terms of unit basis vectors, $\mathbf{F} = \mathbf{k}$, $\mathbf{L} = \mathbf{i}$ and $\mathbf{U} = \mathbf{j}$). However, it's equally common for $+x$ to be *front* and $+z$ to be right ($\mathbf{F} = \mathbf{i}$, $\mathbf{R} = \mathbf{k}$, $\mathbf{U} = \mathbf{j}$). I've also worked with engines in which the z-axis is oriented vertically. The only real requirement is that you stick to one convention consistently throughout your engine.

As an example of how intuitive axis names can reduce confusion, consider *Euler angles* $(pitch, yaw, roll)$, which are often used to describe an aircraft's orientation. It's not possible to define pitch, yaw, and roll angles in terms of the $(\mathbf{i}, \mathbf{j}, \mathbf{k})$ basis vectors because their orientation is arbitrary. However, we *can* define pitch, yaw and roll in terms of the $(\mathbf{L}, \mathbf{U}, \mathbf{F})$ basis vectors, because their orientations are clearly defined. Specifically,

- *pitch* is rotation about **L** or **R**,
- *yaw* is rotation about **U**, and
- *roll* is rotation about **F**.

5.3.9.2 World Space

World space is a fixed coordinate space, in which the positions, orientations and scales of all objects in the game world are expressed. This coordinate space ties all the individual objects together into a cohesive virtual world.

The location of the world-space origin is arbitrary, but it is often placed near the center of the playable game space to minimize the reduction in floating-point precision that can occur when (x, y, z) coordinates grow very large. Likewise, the orientation of the x-, y- and z-axes is arbitrary, although most of the engines I've encountered use either a y-up or a z-up convention. The y-up convention was probably an extension of the two-dimensional convention found in most mathematics textbooks, where the y-axis is shown going up and the x-axis going to the right. The z-up convention is also common,

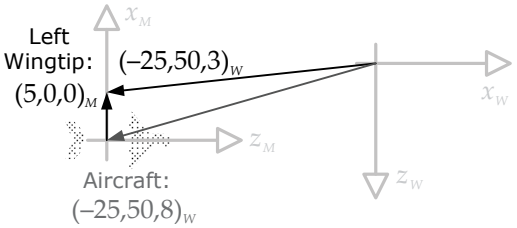

Figure 5.19. A Lear jet whose left wingtip is at $(5, 0, 0)$ in model space. If the jet is rotated by 90 degrees about the world-space y-axis, and its model-space origin translated to $(-25, 50, 8)$ in world space, then its left wingtip would end up at $(-25, 50, 3)$ when expressed in world-space coordinates.

because it allows a top-down orthographic view of the game world to look like a traditional two-dimensional xy-plot.

As an example, let's say that our aircraft's left wingtip is at $(5, 0, 0)$ in model space. (In our game, front vectors correspond to the positive z-axis in model space with y up, as shown in Figure 5.18.) Now imagine that the jet is facing down the positive x-axis in world space, with its model-space origin at some arbitrary location, such as $(-25, 50, 8)$. Because the **F** vector of the airplane, which corresponds to $+z$ in model space, is facing down the $+x$-axis in world space, we know that the jet has been rotated by 90 degrees about the world y-axis. So, if the aircraft were sitting at the world-space origin, its left wingtip would be at $(0, 0, -5)$ in world space. But because the aircraft's origin has been translated to $(-25, 50, 8)$, the final position of the jet's left wingtip in world space is $(-25, 50, [8 - 5]) = (-25, 50, 3)$. This is illustrated in Figure 5.19.

We could of course populate our friendly skies with more than one Lear jet. In that case, all of their left wingtips would have coordinates of $(5, 0, 0)$ in model space. But in world space, the left wingtips would have all sorts of interesting coordinates, depending on the orientation and translation of each aircraft.

5.3.9.3 View Space

View space (also known as *camera space*) is a coordinate frame fixed to the camera. The view space origin is placed at the focal point of the camera. Again, any axis orientation scheme is possible. However, a y-up convention with z increasing in the direction the camera is facing (left-handed) is typical because it allows z coordinates to represent depths into the screen. Other engines and APIs, such as OpenGL, define view space to be right-handed, in which case the camera faces towards *negative* z, and z coordinates represent negative depths.

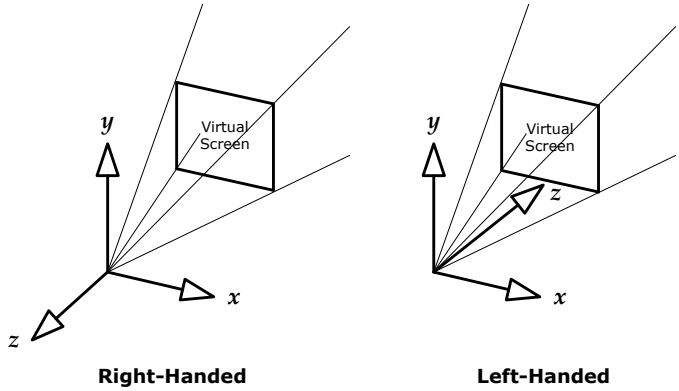

Figure 5.20. Left- and right-handed examples of view space, also known as camera space.

Two possible definitions of view space are illustrated in Figure 5.20.

5.3.10 Change of Basis

In games and computer graphics, it is often quite useful to convert an object's position, orientation and scale from one coordinate system into another. We call this operation a *change of basis*.

5.3.10.1 Coordinate Space Hierarchies

Coordinate frames are relative. That is, if you want to quantify the position, orientation and scale of a set of axes in three-dimensional space, you must specify these quantities *relative to* some other set of axes (otherwise the numbers would have no meaning). This implies that coordinate spaces form a *hierarchy*—every coordinate space is a *child* of some other coordinate space, and the other space acts as its *parent*. World space has no parent; it is at the root of the coordinate-space tree, and all other coordinate systems are ultimately specified relative to it, either as direct children or more-distant relatives.

5.3.10.2 Building a Change of Basis Matrix

The matrix that transforms points and directions from any child coordinate system C to its parent coordinate system P can be written $\mathbf{M}_{C \to P}$ (pronounced "C to P"). The subscript indicates that this matrix transforms points and directions from child space to parent space. Any child-space position vector $\mathbf{P}_C$ can

be transformed into a parent-space position vector $\mathbf{P}_P$ as follows:

$$\begin{aligned}\mathbf{P}_P &= \mathbf{P}_C\mathbf{M}_{C\to P};\\ \mathbf{M}_{C\to P} &= \begin{bmatrix}\mathbf{i}_C & 0\\ \mathbf{j}_C & 0\\ \mathbf{k}_C & 0\\ \mathbf{t}_C & 1\end{bmatrix}\\ &= \begin{bmatrix}i_{Cx} & i_{Cy} & i_{Cz} & 0\\ j_{Cx} & j_{Cy} & j_{Cz} & 0\\ k_{Cx} & k_{Cy} & k_{Cz} & 0\\ t_{Cx} & t_{Cy} & t_{Cz} & 1\end{bmatrix}.\end{aligned}$$

In this equation,

- $\mathbf{i}_C$ is the unit basis vector along the child space x-axis, expressed in parent-space coordinates;
- $\mathbf{j}_C$ is the unit basis vector along the child space y-axis, in parent space;
- $\mathbf{k}_C$ is the unit basis vector along the child space z-axis, in parent space; and
- $\mathbf{t}_C$ is the translation of the child coordinate system relative to parent space.

This result should not be too surprising. The $\mathbf{t}_C$ vector is just the translation of the child-space axes relative to parent space, so if the rest of the matrix were identity, the point $(0, 0, 0)$ in child space would become $\mathbf{t}_C$ in parent space, just as we'd expect. The $\mathbf{i}_C$, $\mathbf{j}_C$ and $\mathbf{k}_C$ unit vectors form the upper 3×3 of the matrix, which is a pure rotation matrix because these vectors are of unit length. We can see this more clearly by considering a simple example, such as a situation in which child space is rotated by an angle γ about the z-axis, with no translation. Recall from Equation (5.6) that the matrix for such a rotation is given by

$$\text{rotate}_z(\mathbf{r}, \gamma) = \begin{bmatrix}r_x & r_y & r_z & 1\end{bmatrix}\begin{bmatrix}\cos\gamma & \sin\gamma & 0 & 0\\ -\sin\gamma & \cos\gamma & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix}.$$

But in Figure 5.21, we can see that the coordinates of the $\mathbf{i}_C$ and $\mathbf{j}_C$ vectors, expressed in parent space, are $\mathbf{i}_C = \begin{bmatrix}\cos\gamma & \sin\gamma & 0\end{bmatrix}$ and $\mathbf{j}_C = \begin{bmatrix}-\sin\gamma & \cos\gamma & 0\end{bmatrix}$. When we plug these vectors into our formula for $\mathbf{M}_{C\to P}$, with $\mathbf{k}_C = \begin{bmatrix}0 & 0 & 1\end{bmatrix}$, it exactly matches the matrix $\text{rotate}_z(\mathbf{r}, \gamma)$ from Equation (5.6).

Scaling the Child Axes

Scaling of the child coordinate system is accomplished by simply scaling the unit basis vectors appropriately. For example, if child space is scaled up by a factor of two, then the basis vectors $\mathbf{i}_C$, $\mathbf{j}_C$ and $\mathbf{k}_C$ will be of length 2 instead of unit length.

5.3.10.3 Extracting Unit Basis Vectors from a Matrix

The fact that we can build a change of basis matrix out of a translation and three Cartesian basis vectors gives us another powerful tool: Given *any* affine 4×4 transformation matrix, we can go in the other direction and extract the child-space basis vectors $\mathbf{i}_C$, $\mathbf{j}_C$ and $\mathbf{k}_C$ from it by simply isolating the appropriate rows of the matrix (or columns if your math library uses column vectors).

This can be incredibly useful. Let's say we are given a vehicle's model-to-world transform as an affine 4×4 matrix (a very common representation). This is really just a change of basis matrix, transforming points in model space into their equivalents in world space. Let's further assume that in our game, the positive z-axis always points in the direction that an object is facing. So, to find a unit vector representing the vehicle's facing direction, we can simply extract $\mathbf{k}_C$ directly from the model-to-world matrix (by grabbing its third row). This vector will already be normalized and ready to go.

5.3.10.4 Transforming Coordinate Systems versus Vectors

We've said that the matrix $\mathbf{M}_{C\to P}$ transforms points and directions from child space into parent space. Recall that the fourth row of $\mathbf{M}_{C\to P}$ contains $\mathbf{t}_C$, the translation of the child coordinate axes relative to the world-space axes. Therefore, another way to visualize the matrix $\mathbf{M}_{C\to P}$ is to imagine it taking the parent *coordinate axes* and transforming them *into* the child axes. This is the

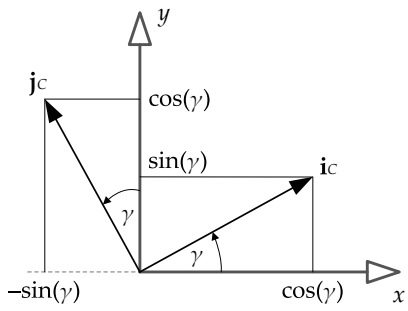

Figure 5.21. Change of basis when child axes are rotated by an angle γ relative to parent.

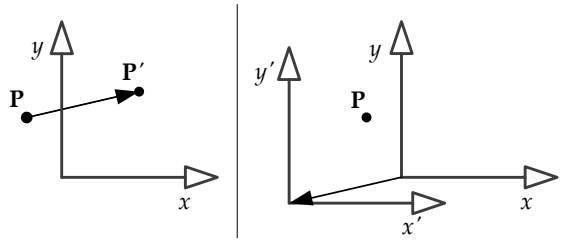

Figure 5.22. Two ways to interpret a transformation matrix. On the left, the point moves against a fixed set of axes. On the right, the axes move in the opposite direction while the point remains fixed.

reverse of what happens to points and direction vectors. In other words, if a matrix transforms *vectors* from child space to parent space, then it also transforms *coordinate axes* from parent space to child space. This makes sense when you think about it—moving a point 20 units to the right with the coordinate axes fixed is the same as moving the coordinate axes 20 units to the left with the point fixed. This concept is illustrated in Figure 5.22.

Of course, this is just another point of potential confusion. If you're thinking in terms of coordinate axes, then transformations go in one direction, but if you're thinking in terms of points and vectors, they go in the other direction! As with many confusing things in life, your best bet is probably to choose a single "canonical" way of thinking about things and stick with it. For example, in this book we've chosen the following conventions:

- Transformations apply to vectors (not coordinate axes).
- Vectors are written as rows (not columns).

Taken together, these two conventions allow us to read sequences of matrix multiplications from left to right and have them make sense (e.g., in the expression $\mathbf{r}_D = \mathbf{r}_A\mathbf{M}_{A\to B}\mathbf{M}_{B\to C}\mathbf{M}_{C\to D}$, the B's and C's in effect "cancel out," leaving only $\mathbf{r}_D = \mathbf{r}_A\mathbf{M}_{A\to D}$). Obviously if you start thinking about the coordinate axes moving around rather than the points and vectors, you either have to read the transforms from right to left, or flip one of these two conventions around. It doesn't really matter what conventions you choose as long as *you* find them easy to remember and work with.

That said, it's important to note that certain problems are easier to think about in terms of vectors being transformed, while others are easier to work with when you imagine the coordinate axes moving around. Once you get good at thinking about 3D vector and matrix math, you'll find it pretty easy to flip back and forth between conventions as needed to suit the problem at hand.

5.3.11 Transforming Normal Vectors

A *normal vector* is a special kind of vector, because in addition to (usually!) being of unit length, it carries with it the additional requirement that it should always remain *perpendicular* to whatever surface or plane it is associated with. Special care must be taken when transforming a normal vector to ensure that both its length and perpendicularity properties are maintained.

In general, if a point or (non-normal) vector can be rotated from space A to space B via the 3×3 matrix $\mathbf{M}_{A \to B}$, then a normal vector $\mathbf{n}$ will be transformed from space A to space B via the *inverse transpose* of that matrix, $(\mathbf{M}_{A \to B}^{-1})^{\mathrm{T}}$. We will not prove or derive this result here (see [32, Section 3.5] for an excellent derivation). However, we will observe that if the matrix $\mathbf{M}_{A \to B}$ contains only uniform scale and no shear, then the angles between all surfaces and vectors in space B will be the same as they were in space A. In this case, the matrix $\mathbf{M}_{A \to B}$ will actually work just fine for any vector, normal or non-normal. However, if $\mathbf{M}_{A \to B}$ contains nonuniform scale or shear (i.e., is *non-orthogonal*), then the angles between surfaces and vectors are *not* preserved when moving from space A to space B. A vector that was normal to a surface in space A will not necessarily be perpendicular to that surface in space B. The inverse transpose operation accounts for this distortion, bringing normal vectors back into perpendicularity with their surfaces even when the transformation involves nonuniform scale or shear. Another way of looking at this is that the inverse transpose is required because a surface normal is really a *pseudovector* rather than a regular vector (see Section 5.2.4.9).

5.3.12 Storing Matrices in Memory

In the C and C++ languages, a two-dimensional array is often used to store a matrix. Recall that in C/C++ two-dimensional array syntax, the first subscript is the row and the second is the column, and the column index varies fastest as you move through memory sequentially.

```
float m[4][4]; // [row][col], col varies fastest

// "flatten" the array to demonstrate ordering
float* pm = &m[0][0];
ASSERT( &pm[0] == &m[0][0] );
ASSERT( &pm[1] == &m[0][1] );
ASSERT( &pm[2] == &m[0][2] );
// etc.
```

We have two choices when storing a matrix in a two-dimensional C/C++ array. We can either

1. store the vectors ($\mathbf{i}_C$, $\mathbf{j}_C$, $\mathbf{k}_C$, $\mathbf{t}_C$) *contiguously* in memory (i.e., each row contains a single vector), or

2. store the vectors *strided* in memory (i.e., each column contains one vector).

The benefit of approach (1) is that we can address any one of the four vectors by simply indexing into the matrix and interpreting the four contiguous values we find there as a 4-element vector. This layout also has the benefit of matching up exactly with *row vector* matrix equations (which is another reason why I've selected row vector notation for this book). Approach (2) is sometimes necessary when doing fast matrix-vector multiplies using a vector-enabled (SIMD) microprocessor, as we'll see later in this chapter. In most game engines I've personally encountered, matrices are stored using approach (1), with the *vectors* in the *rows* of the two-dimensional C/C++ array. This is shown below:

```
float M[4][4];

M[0][0]=ix;  M[0][1]=iy;  M[0][2]=iz;  M[0][3]=0.0f;
M[1][0]=jx;  M[1][1]=jy;  M[1][2]=jz;  M[1][3]=0.0f;
M[2][0]=kx;  M[2][1]=ky;  M[2][2]=kz;  M[2][3]=0.0f;
M[3][0]=tx;  M[3][1]=ty;  M[3][2]=tz;  M[3][3]=1.0f;
```

The matrix looks like this when viewed in a debugger:

```
M[][]
    [0]
        [0] ix
        [1] iy
        [2] iz
        [3] 0.0000
    [1]
        [0] jx
        [1] jy
        [2] jz
        [3] 0.0000
    [2]
        [0] kx
        [1] ky
        [2] kz
        [3] 0.0000
    [3]
        [0] tx
        [1] ty
        [2] tz
        [3] 1.0000
```

One easy way to determine which layout your engine uses is to find a function that builds a 4×4 translation matrix. (Every good 3D math library provides such a function.) You can then inspect the source code to see where the elements of the **t** vector are being stored. If you don't have access to the source code of your math library (which is pretty rare in the game industry), you can always call the function with an easy-to-recognize translation like $(4, 3, 2)$, and then inspect the resulting matrix. If row 3 contains the values `4.0f`, `3.0f`, `2.0f`, `1.0f`, then the vectors are in the rows, otherwise the vectors are in the columns.

5.4 Quaternions

We've seen that a 3×3 matrix can be used to represent an arbitrary rotation in three dimensions. However, a matrix is not always an ideal representation of a rotation, for a number of reasons:

1. We need nine floating-point values to represent a rotation, which seems excessive considering that we only have three degrees of freedom—pitch, yaw and roll.

2. Rotating a vector requires a vector-matrix multiplication, which involves three dot products, or a total of nine multiplications and six additions. We would like to find a rotational representation that is less expensive to calculate, if possible.

3. In games and computer graphics, it's often important to be able to find rotations that are some percentage of the way between two known rotations. For example, if we are to smoothly animate a camera from some starting orientation A to some final orientation B over the course of a few seconds, we need to be able to find lots of intermediate rotations between A and B over the course of the animation. It turns out to be difficult to do this when the A and B orientations are expressed as matrices.

Thankfully, there is a rotational representation that overcomes these three problems. It is a mathematical object known as a *quaternion*. A quaternion looks a lot like a four-dimensional vector, but it *behaves* quite differently. We usually write quaternions using non-italic, non-boldface type, like this: $\mathrm{q} = \begin{bmatrix} q_x & q_y & q_z & q_w \end{bmatrix}$.

Quaternions were developed by Sir William Rowan Hamilton in 1843 as an extension to the complex numbers. (Specifically, a quaternion may be interpreted as a four-dimensional complex number, with a single real axis

and three imaginary axes represented by the imaginary numbers i, j and k. As such, a quaternion can be written in "complex form" as follows: $\mathsf{q} = iq_x + jq_y + kq_z + q_w$.) Quaternions were first used to solve problems in the area of mechanics. Technically speaking, a quaternion obeys a set of rules known as a four-dimensional *normed division algebra* over the real numbers. Thankfully, we won't need to understand the details of these rather esoteric algebraic rules. For our purposes, it will suffice to know that the *unit-length* quaternions (i.e., all quaternions obeying the constraint $q_x^2 + q_y^2 + q_z^2 + q_w^2 = 1$) represent three-dimensional rotations.

There are a lot of great papers, web pages and presentations on quaternions available on the web for further reading. Here's one of my favorites: http://graphics.ucsd.edu/courses/cse169_w05/CSE169_04.ppt.

5.4.1 Unit Quaternions as 3D Rotations

A unit quaternion can be visualized as a three-dimensional vector plus a fourth scalar coordinate. The vector part $\mathbf{q}_V$ is the unit axis of rotation, scaled by the sine of the half-angle of the rotation. The scalar part q_S is the cosine of the half-angle. So the unit quaternion q can be written as follows:

$$\begin{aligned}\mathsf{q} &= \begin{bmatrix}\mathbf{q}_V & q_S\end{bmatrix} \\ &= \begin{bmatrix}\mathbf{a}\sin\frac{\theta}{2} & \cos\frac{\theta}{2}\end{bmatrix},\end{aligned}$$

where $\mathbf{a}$ is a unit vector along the axis of rotation, and θ is the angle of rotation. The direction of the rotation follows the *right-hand rule*, so if your thumb points in the direction of $\mathbf{a}$, positive rotations will be in the direction of your curved fingers.

Of course, we can also write q as a simple four-element vector:

$$\mathsf{q} = \begin{bmatrix}q_x & q_y & q_z & q_w\end{bmatrix}, \text{where}$$

$$\begin{aligned}q_x &= q_{V_x} = a_x \sin\tfrac{\theta}{2}, \\ q_y &= q_{V_y} = a_y \sin\tfrac{\theta}{2}, \\ q_z &= q_{V_z} = a_z \sin\tfrac{\theta}{2}, \\ q_w &= q_S = \cos\tfrac{\theta}{2}.\end{aligned}$$

A unit quaternion is very much like an axis+angle representation of a rotation (i.e., a four-element vector of the form $\begin{bmatrix}\mathbf{a} & \theta\end{bmatrix}$). However, quaternions are more convenient mathematically than their axis+angle counterparts, as we shall see below.

5.4.2 Quaternion Operations

Quaternions support some of the familiar operations from vector algebra, such as magnitude and vector addition. However, we must remember that the sum of two unit quaternions does not represent a 3D rotation, because such a quaternion would not be of unit length. As a result, you won't see any quaternion sums in a game engine, unless they are scaled in some way to preserve the unit length requirement.

5.4.2.1 Quaternion Multiplication

One of the most important operations we will perform on quaternions is that of multiplication. Given two quaternions $\mathtt{p}$ and $\mathtt{q}$ representing two rotations $\mathbf{P}$ and $\mathbf{Q}$, respectively, the product $\mathtt{pq}$ represents the composite rotation (i.e., rotation $\mathbf{Q}$ followed by rotation $\mathbf{P}$). There are actually quite a few different kinds of quaternion multiplication, but we'll restrict this discussion to the variety used in conjunction with 3D rotations, namely the Grassman product. Using this definition, the product $\mathtt{pq}$ is defined as follows:

$$\mathtt{pq} = \left[(p_S\mathbf{q}_V + q_S\mathbf{p}_V + \mathbf{p}_V \times \mathbf{q}_V) \quad (p_S q_S - \mathbf{p}_V \cdot \mathbf{q}_V)\right].$$

Notice how the Grassman product is defined in terms of a vector part, which ends up in the x, y and z components of the resultant quaternion, and a scalar part, which ends up in the w component.

5.4.2.2 Conjugate and Inverse

The *inverse* of a quaternion $\mathtt{q}$ is denoted $\mathtt{q}^{-1}$ and is defined as a quaternion that, when multiplied by the original, yields the scalar 1 (i.e., $\mathtt{q}\mathtt{q}^{-1} = 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k} + 1$). The quaternion $\begin{bmatrix}0 & 0 & 0 & 1\end{bmatrix}$ represents a zero rotation (which makes sense since $\sin(0) = 0$ for the first three components, and $\cos(0) = 1$ for the last component).

In order to calculate the inverse of a quaternion, we must first define a quantity known as the *conjugate*. This is usually denoted $\mathtt{q}^*$ and it is defined as follows:

$$\mathtt{q}^* = \begin{bmatrix}-\mathbf{q}_V & q_S\end{bmatrix}.$$

In other words, we negate the vector part but leave the scalar part unchanged.

Given this definition of the quaternion conjugate, the inverse quaternion $\mathtt{q}^{-1}$ is defined as follows:

$$\mathtt{q}^{-1} = \frac{\mathtt{q}^*}{|\mathtt{q}|^2}.$$

Our quaternions are always of unit length (i.e., $|\mathsf{q}| = 1$), because they represent 3D rotations. So, for our purposes, the inverse and the conjugate are identical:

$$\mathsf{q}^{-1} = \mathsf{q}^* = \begin{bmatrix} -\mathbf{q}_V & q_S \end{bmatrix} \qquad \text{when} \qquad |\mathsf{q}| = 1.$$

This fact is incredibly useful, because it means we can always avoid doing the (relatively expensive) division by the squared magnitude when inverting a quaternion, as long as we know a priori that the quaternion is normalized. This also means that inverting a quaternion is generally much faster than inverting a 3×3 matrix—a fact that you may be able to leverage in some situations when optimizing your engine.

Conjugate and Inverse of a Product

The conjugate of a quaternion product (pq) is equal to the reverse product of the conjugates of the individual quaternions:

$$(\mathsf{pq})^* = \mathsf{q}^*\mathsf{p}^*.$$

Likewise, the inverse of a quaternion product is equal to the reverse product of the inverses of the individual quaternions:

$$(\mathsf{pq})^{-1} = \mathsf{q}^{-1}\mathsf{p}^{-1}. \tag{5.8}$$

This is analogous to the reversal that occurs when transposing or inverting matrix products.

5.4.3 Rotating Vectors with Quaternions

How can we apply a quaternion rotation to a vector? The first step is to rewrite the vector in *quaternion form*. A vector is a sum involving the unit basis vectors **i**, **j** and **k**. A quaternion is a sum involving **i**, **j** and **k**, but with a fourth scalar term as well. So it makes sense that a vector can be written as a quaternion with its scalar term q_S equal to zero. Given the vector $\mathbf{v}$, we can write a corresponding quaternion $\mathsf{v} = \begin{bmatrix} \mathbf{v} & 0 \end{bmatrix} = \begin{bmatrix} v_x & v_y & v_z & 0 \end{bmatrix}$.

In order to rotate a vector $\mathbf{v}$ by a quaternion q, we premultiply the vector (written in its quaternion form v) by q and then post-multiply it by the *inverse* quaternion q^{-1}. Therefore, the rotated vector $\mathbf{v}'$ can be found as follows:

$$\mathsf{v}' = \text{rotate}(\mathsf{q}, \mathbf{v}) = \mathsf{q}\mathsf{v}\mathsf{q}^{-1}.$$

This is equivalent to using the quaternion conjugate, because our quaternions are always unit length:

$$\mathsf{v}' = \text{rotate}(\mathsf{q}, \mathbf{v}) = \mathsf{q}\mathsf{v}\mathsf{q}^*. \tag{5.9}$$

The rotated vector $\mathbf{v}'$ is obtained by simply extracting it from its quaternion form v'.

Quaternion multiplication can be useful in all sorts of situations in real games. For example, let's say that we want to find a unit vector describing the direction in which an aircraft is flying. We'll further assume that in our game, the positive z-axis always points toward the front of an object by convention. So the forward unit vector of any object *in model space* is always $\mathbf{F}_M \equiv \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$ by definition. To transform this vector into world space, we can simply take our aircraft's orientation quaternion q and use it with Equation (5.9) to rotate our model-space vector $\mathbf{F}_M$ into its world-space equivalent $\mathbf{F}_W$ (after converting these vectors into quaternion form, of course):

$$F_W = qF_Mq^{-1} = q \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix} q^{-1}.$$

5.4.3.1 Quaternion Concatenation

Rotations can be *concatenated* in exactly the same way that matrix-based transformations can, by multiplying the quaternions together. For example, consider three distinct rotations, represented by the quaternions q_1, q_2 and q_3, with matrix equivalents $\mathbf{R}_1$, $\mathbf{R}_2$ and $\mathbf{R}_3$. We want to apply rotation 1 first, followed by rotation 2 and finally rotation 3. The composite rotation matrix $\mathbf{R}_{\text{net}}$ can be found and applied to a vector $\mathbf{v}$ as follows:

$$\begin{aligned} \mathbf{R}_{\text{net}} &= \mathbf{R}_1\mathbf{R}_2\mathbf{R}_3; \\ \mathbf{v}' &= \mathbf{v}\mathbf{R}_1\mathbf{R}_2\mathbf{R}_3 \\ &= \mathbf{v}\mathbf{R}_{\text{net}}. \end{aligned}$$

Likewise, the composite rotation quaternion q_{net} can be found and applied to vector $\mathbf{v}$ (in its quaternion form, v) as follows:

$$\begin{aligned} q_{\text{net}} &= q_3q_2q_1; \\ v' &= q_3q_2q_1 \, v \, q_1^{-1}q_2^{-1}q_3^{-1} \\ &= q_{\text{net}} \, v \, q_{\text{net}}^{-1}. \end{aligned}$$

Notice how the quaternion product must be performed in an order *opposite* to that in which the rotations are applied ($q_3q_2q_1$). This is because quaternion rotations always multiply on *both* sides of the vector, with the uninverted quaternions on the left and the inverted quaternions on the right. As we saw in Equation (5.8), the inverse of a quaternion product is the reverse product of the individual inverses, so the uninverted quaternions read right-to-left while the inverted quaternions read left-to-right.

5.4.4 Quaternion-Matrix Equivalence

We can convert any 3D rotation freely between a 3×3 matrix representation $\mathbf{R}$ and a quaternion representation q. If we let $\mathtt{q} = [\mathbf{q}_V \;\; q_S] = [q_{Vx} \;\; q_{Vy} \;\; q_{Vz} \;\; q_S] = \begin{bmatrix} x & y & z & w \end{bmatrix}$, then we can find $\mathbf{R}$ as follows:

$$\mathbf{R} = \begin{bmatrix} 1-2y^2-2z^2 & 2xy+2zw & 2xz-2yw \\ 2xy-2zw & 1-2x^2-2z^2 & 2yz+2xw \\ 2xz+2yw & 2yz-2xw & 1-2x^2-2y^2 \end{bmatrix}.$$

Likewise, given $\mathbf{R}$, we can find q as follows (where q[0] = q_{Vx}, q[1] = q_{Vy}, q[2] = q_{Vz} and q[3] = q_S). This code assumes that we are using row vectors in C/C++ (i.e., that the rows of the matrix correspond to the rows of the matrix $\mathbf{R}$ shown above). The code was adapted from a *Gamasutra* article by Nick Bobic, published on July 5, 1998, which is available here: http://www.gamasutra.com/view/feature/3278/rotating_objects_using_quaternions.php. For a discussion of some even faster methods for converting a matrix to a quaternion, leveraging various assumptions about the nature of the matrix, see http://www.euclideanspace.com/maths/geometry/rotations/conversions/matrixToQuaternion/index.htm.

```
void matrixToQuaternion(
        const float  R[3][3],
        float        q[/*4*/])
{
   float trace = R[0][0] + R[1][1] + R[2][2];

   // check the diagonal
   if (trace > 0.0f)
   {
      float s = sqrt(trace + 1.0f);
      q[3] = s * 0.5f;

      float t = 0.5f / s;
      q[0] = (R[2][1] - R[1][2]) * t;
      q[1] = (R[0][2] - R[2][0]) * t;
      q[2] = (R[1][0] - R[0][1]) * t;
   }
   else
   {
      // diagonal is negative
      int i = 0;
      if (R[1][1] > R[0][0]) i = 1;
      if (R[2][2] > R[i][i]) i = 2;

      static const int NEXT[3] = {1, 2, 0};
```

```
            int j = NEXT[i];
            int k = NEXT[j];

            float s = sqrt((R[i][j]
                           - (R[j][j] + R[k][k]))
                             + 1.0f);

            q[i] = s * 0.5f;

            float t;
            if (s != 0.0)  t = 0.5f / s;
            else           t = s;

            q[3] = (R[k][j] - R[j][k]) * t;
            q[j] = (R[j][i] + R[i][j]) * t;
            q[k] = (R[k][i] + R[i][k]) * t;
        }
    }
```

Let's pause for a moment to consider notational conventions. In this book, we write our quaternions like this: $[\,x\ y\ z\ w\,]$. This differs from the $[\,w\ x\ y\ z\,]$ convention found in many academic papers on quaternions as an extension of the complex numbers. Our convention arises from an effort to be consistent with the common practice of writing homogeneous vectors as $[\,x\ y\ z\ 1\,]$ (with the $w = 1$ at the end). The academic convention arises from the parallels between quaternions and complex numbers. Regular two-dimensional complex numbers are typically written in the form $c = a + jb$, and the corresponding quaternion notation is $q = w + ix + jy + kz$. So be careful out there—make sure you know which convention is being used before you dive into a paper head first!

5.4.5 Rotational Linear Interpolation

Rotational interpolation has many applications in the animation, dynamics and camera systems of a game engine. With the help of quaternions, rotations can be easily interpolated just as vectors and points can.

The easiest and least computationally intensive approach is to perform a four-dimensional vector LERP on the quaternions you wish to interpolate. Given two quaternions q_A and q_B representing rotations A and B, we can find an intermediate rotation $\mathrm{q}_{\mathrm{LERP}}$ that is β percent of the way from A to B as fol-

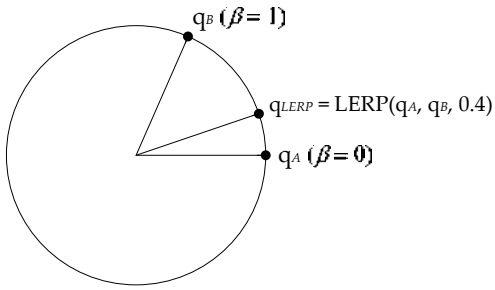

Figure 5.23. Linear interpolation (LERP) between two quaternions $\mathbf{q}_A$ and $\mathbf{q}_B$.

lows:

$$\begin{aligned}\mathbf{q}_{\text{LERP}} = \text{LERP}(\mathbf{q}_A, \mathbf{q}_B, \beta) &= \frac{(1-\beta)\mathbf{q}_A + \beta\mathbf{q}_B}{|(1-\beta)\mathbf{q}_A + \beta\mathbf{q}_B|} \\ &= \text{normalize}\left(\begin{bmatrix}(1-\beta)q_{Ax} + \beta q_{Bx} \\ (1-\beta)q_{Ay} + \beta q_{By} \\ (1-\beta)q_{Az} + \beta q_{Bz} \\ (1-\beta)q_{Aw} + \beta q_{Bw}\end{bmatrix}^{\text{T}}\right).\end{aligned}$$

Notice that the resultant interpolated quaternion had to be renormalized. This is necessary because the LERP operation does not preserve a vector's length in general.

Geometrically, $\mathbf{q}_{\text{LERP}} = \text{LERP}(\mathbf{q}_A, \mathbf{q}_B, \beta)$ is the quaternion whose orientation lies β percent of the way from orientation A to orientation B, as shown (in two dimensions for clarity) in Figure 5.23. Mathematically, the LERP operation results in a *weighted average* of the two quaternions, with weights $(1-\beta)$ and β (notice that these two weights sum to 1).

5.4.5.1 Spherical Linear Interpolation

The problem with the LERP operation is that it does not take account of the fact that quaternions are really points on a four-dimensional *hypersphere*. A LERP effectively interpolates along a *chord* of the hypersphere, rather than along the surface of the hypersphere itself. This leads to rotation animations that do not have a constant angular speed when the parameter β is changing at a constant rate. The rotation will appear slower at the end points and faster in the middle of the animation.

To solve this problem, we can use a variant of the LERP operation known as *spherical linear interpolation*, or SLERP for short. The SLERP operation uses sines and cosines to interpolate along a *great circle* of the 4D hypersphere,

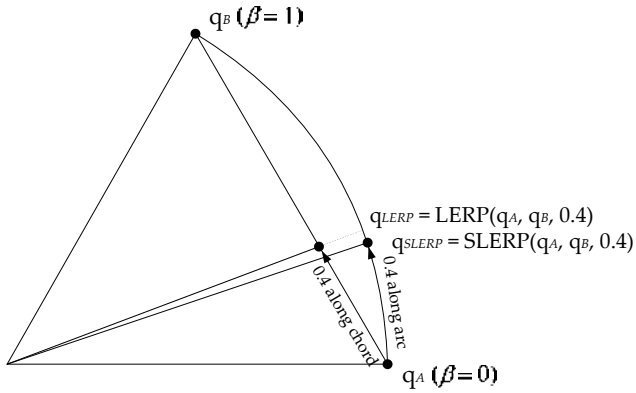

Figure 5.24. Spherical linear interpolation along a great circle arc of a 4D hypersphere.

rather than along a chord, as shown in Figure 5.24. This results in a constant angular speed when β varies at a constant rate.

The formula for SLERP is similar to the LERP formula, but the weights $(1-\beta)$ and β are replaced with weights w_p and w_q involving sines of the angle between the two quaternions.

$$\text{SLERP}(\mathsf{p}, \mathsf{q}, \beta) = w_p \mathsf{p} + w_q \mathsf{q},$$

where

$$w_p = \frac{\sin(1-\beta)\theta}{\sin\theta},$$
$$w_q = \frac{\sin\beta\theta}{\sin\theta}.$$

The cosine of the angle between any two unit-length quaternions can be found by taking their four-dimensional dot product. Once we know $\cos\theta$, we can calculate the angle θ and the various sines we need quite easily:

$$\cos\theta = \mathsf{p}\cdot\mathsf{q} = p_x q_x + p_y q_y + p_z q_z + p_w q_w;$$
$$\theta = \cos^{-1}(\mathsf{p}\cdot\mathsf{q}).$$

5.4.5.2 To SLERP or Not to SLERP (That's Still the Question)

The jury is still out on whether or not to use SLERP in a game engine. Jonathan Blow wrote a great article positing that SLERP is too expensive, and LERP's quality is not really that bad—therefore, he suggests, we should understand SLERP but avoid it in our game engines (see http://number-none.com/product/Understanding%20Slerp,%20Then%20Not%20Using%20It/index.html).